

JJBike

1. Objetivo.

Construir um armazém de dados para que o administrador da empresa fictícia JJBike possa compreender seu negócio.

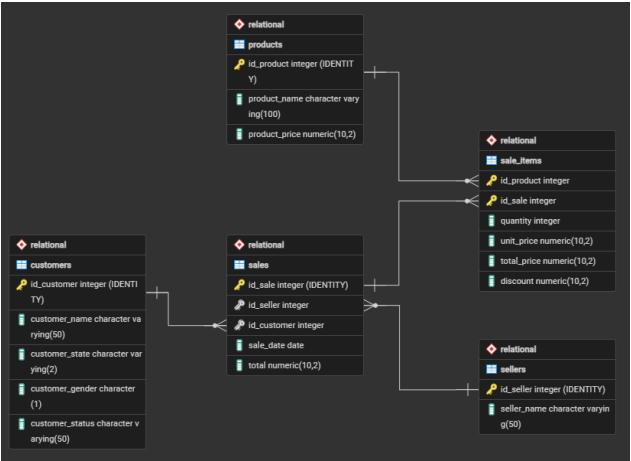
2. Modelagem.

Para criar o modelo da estrutura do banco de dados analítico (OLAP) considerando o banco de dados transacional (OLTP), o assunto que se quer analisar (o fato) e as dimensões são os diversos pontos de vista sobre o qual se quer analisar o fato (as dimensões), utilizando o modelo Star Schema (modelo mais comum para Business Intelligence), foram feitas as seguintes transformações:

Dimensão	Atributos	Tabela de origem
dim_product (o quê?)	product_key (SK)	-
	id_product	product
	product_name	product
	validity_start_date (versionamento)	-
	validity_end_date (versionamento)	-
dim_seller (quem?)	seller_key (SK)	-
	id_seller	seller
	seller_name	seller
	validity_start_date (versionamento)	-
	validity_end_date (versionamento)	-
dim_customer (para quem?)	customer_key (SK)	-
	id_customer	customer
	customer_name	customer
	customer_state	customer
	customer_gender	customer
	customer_status	customer
	validity_start_date	-

	(versionamento)	
	validity_end_date (versionamento)	-
dim_time (quando?)	time_key (SK)	-
	date	-
	time_day	-
	time_month	-
	time_year	-
	time_weekday	-
	time_quarter	-

Fato	Tabela consolidada	Granularidade	Atributos	Tabela de origem
fact_sales	sales	Item de produto por venda.	sale_key (SK)	-
			seller_key (FK)	dim_seller
			customer_key (FK)	dim_customer
			product_key	dim_product
			time_key (FK)	dim_time
			quantity	sale_items
			unit_price	sale_items
			total_price	sale_items
			discount	sale_items



No pgAdmin 4, foi criado o banco de dados 'JJBike'.

- Obs.: No desenvolvimento do código SQL, será utilizado o padrão de nomenclatura Snake Case.

4. Ambiente transacional (OLTP) - pgAdmin 4.

4.1. Criação do schema relacional.

Foi criado o schema Relational, que serve como camada de origem dos dados transacionais da empresa JJBike. Dentro dele, o arquivo 1. Scripts_Create_Table_Relational.sql criou cinco tabelas operacionais. Em todas elas, a chave primária é gerada automaticamente pela cláusula GENERATED ALWAYS AS IDENTITY, eliminando a necessidade de controlar sequências externas. Todas as colunas, exceto as próprias chaves primárias, foram definidas com NOT NULL, garantindo que nenhum registro seja persistido com dados obrigatórios em aberto.

- **sellers:** Armazena os dados dos vendedores. O atributo seller_name identifica o nome do vendedor e é o único campo além da PK, refletindo o escopo intencional desta entidade no modelo transacional;
- **products:** Armazena o catálogo de produtos. O atributo product_name identifica o produto e product_price registra o preço de tabela vigente ao momento do cadastro. Ambos são obrigatórios, pois um produto sem nome ou sem preço não pode participar de um processo de venda;
- **customers:** Armazena o cadastro de clientes. O atributo customer_name identifica o cliente; customer_state registra a UF de origem (Varchar(2)); customer_gender registra o gênero em um único caractere (Char(1)); e customer_status registra o nível do plano do cliente, atributo que, por ser mutável ao longo do tempo, será o principal elemento rastreado pelo SCD Tipo 2 no ambiente analítico;
- **sales:** Representa o cabeçalho de cada venda. O atributo id_seller é uma FK que referencia Relational.sellers, e id_customer é uma FK que referencia Relational.customers, ambas obrigatórias, pois uma venda não pode existir sem um vendedor e um cliente associados. O atributo sale_date registra a data em que a transação ocorreu, e total armazena o valor consolidado da venda;
- **sale_items:** Tabela de ligação entre sales e products, com chave primária composta por id_product e id_sale. Foram aplicadas duas políticas de exclusão distintas para preservar a integridade referencial em direções opostas:
 - ON DELETE RESTRICT em id_product: bloqueia a exclusão de um registro em Relational.products enquanto ele estiver referenciado em algum item de venda, impedindo que dados históricos percam a referência ao produto transacionado;
 - ON DELETE CASCADE em id_sale: ao se excluir um registro em Relational.sales, todos os itens correspondentes em sale_items são automaticamente removidos, evitando itens órfãos de uma venda inexistente;
 - Os atributos quantity, unit_price, total_price e discount são as métricas de detalhe de cada item e foram declaradas NOT NULL pois alimentarão diretamente as colunas de medida da tabela fato no ambiente analítico.

4.2. População das tabelas

Os dados foram inseridos por meio dos arquivos 2. Insert_Into_Customers.sql, 3. Insert_Into_Products.sql, 4. Insert_Into_Sellers.sql, 5. Insert_Into_Sales.sql e 6. Insert_Into_SaleItems.sql. Cada arquivo executa comandos INSERT INTO na respectiva tabela do schema Relational, informando apenas as colunas de dados, as chaves primárias são omitidas porque o mecanismo GENERATED ALWAYS AS IDENTITY as atribui automaticamente e de forma sequencial a cada registro inserido.

5. Ambiente de Staging (invisível/conceitual).

Não houve tratamento ou limpeza de dados brutos antes de criar as dimensões, a própria transferência direta de dados entre os schemas do PostgreSQL funcionou como o gatilho de carga.

6. Ambiente analítico (OLAP) - pgAdmin 4.

6.1. Criação do schema e das tabelas dimensionais.

Foi criado o schema Dimensional, que representa o Data Warehouse estruturado em modelo Star Schema. O arquivo 1. Scripts_Create_Table_Dimensional.sql criou as dimensões dim_seller, dim_customer, dim_product e dim_time, além da tabela fato fact_sales. Em todas as tabelas, a Surrogate Key (SK) é gerada automaticamente por GENERATED ALWAYS AS IDENTITY, servindo como chave primária interna do DW, desacoplada dos IDs de origem do sistema transacional.

Todas as colunas foram definidas com NOT NULL, com a única exceção de validity_end_date nas dimensões com SCD Tipo 2. Esse campo permanece NULL enquanto o registro estiver ativo; só é preenchido com a data corrente no momento em que uma mudança de atributo é detectada e uma nova versão do registro precisa ser criada.

As tabelas dimensionais possuem a seguinte estrutura de atributos:

- **dim_seller:** seller_key é a SK gerada automaticamente e atua como PK interna do DW. id_seller preserva o ID de origem vindo de Relational.sellers, mantendo rastreabilidade entre os dois ambientes. seller_name carrega o nome do vendedor. validity_start_date recebe current_date no momento em que o registro é inserido, marcando o início de sua vigência. validity_end_date permanece NULL enquanto o registro for a versão ativa;
- **dim_customer:** customer_key é a SK, PK interna do DW. id_customer preserva o ID de origem. customer_name, customer_state, customer_gender e customer_status são os atributos descritivos rastreados, qualquer alteração em qualquer um deles aciona o versionamento SCD Tipo 2. validity_start_date e validity_end_date controlam o período de vigência de cada versão do registro;
- **dim_product:** product_key é a SK. id_product preserva o ID de origem. product_name é o único atributo descritivo rastreado, uma alteração em seu valor dispara o versionamento. validity_start_date e validity_end_date funcionam da mesma forma que nas demais dimensões SCD Tipo 2;
- **dim_time:** time_key é a SK. time_date armazena a data completa e serve como campo de ligação com o sistema transacional durante a carga da fato. time_day, time_month, time_year e time_quarter são derivados de time_date e habilitam agrupamentos analíticos por granularidade temporal. time_weekday armazena o dia da semana como inteiro (0 = domingo, 6 = sábado). Esta dimensão não possui campos de validade pois o tempo é imutável;
- **fact_sales:** sale_key é a SK da fato. seller_key, customer_key, product_key e time_key são as FKs que referenciam as respectivas dimensões, todas declaradas NOT NULL, pois cada fato precisa obrigatoriamente estar associado a todas as dimensões do modelo. quantity,

unit_price, total_price e discount são as métricas de cada item vendido, também NOT NULL.

6.2. População da dimensão tempo.

A dimensão dim_time foi populada pelo arquivo 2. Insert_Into_dim_time.sql. A estratégia adotada foi a geração programática de um intervalo contínuo de datas de 1º de janeiro de 1970 a 31 de dezembro de 2049, cobrindo 29.220 registros (índices 0 a 29.219). O script é composto por três blocos encadeados:

6.3. Extração (E) e carregamento (L) dos dados do ambiente transacional (OLTP) para o ambiente analítico (OLAP).

As cargas foram realizadas pelo arquivo 3. Script.sql, estruturado em fases sequenciais para demonstrar o funcionamento completo do SCD Tipo 2.

6.3.1. Carga inicial das dimensões.

O mesmo padrão de CTE dupla foi aplicado às três dimensões com suporte a SCD Tipo 2. O mecanismo opera em três etapas encadeadas.

- **CTE S:** Seleciona todos os registros da tabela de origem no schema Relational, por exemplo, Relational.customers. Esta CTE representa o estado atual da fonte transacional e serve como base de comparação para as etapas seguintes;
- **CTE UPD:** Executa um UPDATE na dimensão de destino, por exemplo, Dimensional.dim_customer. O filtro T.id_customer = S.id_customer AND T.validity_end_date IS NULL localiza, para cada ID de origem, o único registro ainda ativo na dimensão, aquele cuja validity_end_date ainda é NULL. Uma segunda condição verifica se algum atributo rastreado diverge entre a fonte e a dimensão (customer_name, customer_state, customer_gender ou customer_status). Quando uma diferença é detectada, validity_end_date recebe current_date, encerrando a vigência daquela versão do registro. A cláusula RETURNING T.id_customer devolve os IDs dos registros recém-encerrados para uso na etapa seguinte;
- **INSERT final:** Insere novos registros na dimensão. A condição WHERE captura dois grupos: (1) os IDs retornados pela CTE UPD, que tiveram uma versão encerrada e precisam de uma nova entrada ativa,, e (2) IDs que ainda não existem na dimensão, registros completamente novos. Em ambos os casos, validity_start_date recebe current_date, marcando o início da vigência da nova versão, e validity_end_date recebe NULL, sinalizando que este é o registro atualmente válido.

6.3.2. Carga da tabela fato (mês a mês).

A tabela fact_sales é populada por um INSERT INTO ... SELECT que consolida dados do schema Relational com as dimensões do schema Dimensional. As vendas foram carregadas mês a mês, janeiro, fevereiro, março e demais meses, para viabilizar a simulação e verificação do SCD Tipo 2 entre os carregamentos.

O bloco SELECT opera com o seguinte mapeamento de atributos e chaves:

- **Métricas:** quantity, unit_price, total_price e discount são extraídos diretamente de Relational.sale_items, pois representam os dados de detalhe de cada item vendido, a granularidade da tabela fato;
- **seller_key:** Obtida pelo INNER JOIN entre Relational.sales e Dimensional.dim_seller usando v.id_seller = vdd.id_seller AND vdd.validity_end_date IS NULL. O filtro validity_end_date IS NULL garante que a fato registre a SK da versão do vendedor que estava ativa no momento da carga, e não de uma versão histórica encerrada;

- **customer_key:** Obtida pelo INNER JOIN entre Relational.sales e Dimensional.dim_customer usando v.id_customer = c.id_customer AND c.validity_end_date IS NULL. O filtro validity_end_date IS NULL é o mecanismo central do SCD Tipo 2 na carga da fato: ele garante que a SK registrada aponte para o customer_status vigente no momento em que aquele mês foi carregado, e não para versões anteriores ou posteriores do cliente;
- **product_key:** Obtida pelo INNER JOIN entre Relational.sale_items e Dimensional.dim_product usando iv.id_product = p.id_product AND p.validity_end_date IS NULL. O mesmo filtro validity_end_date IS NULL captura apenas a versão ativa do produto no momento da carga;
- **time_key:** Obtida pelo INNER JOIN entre Relational.sales e Dimensional.dim_time usando v.sale_date = t.time_date. O campo sale_date da tabela transacional é comparado diretamente ao campo time_date da dimensão, localizando o registro que representa exatamente o dia em que a venda ocorreu. Como cada data possui exatamente um registro em dim_time, não há risco de duplicidade nessa junção;
- **Filtro temporal:** A função date_part('MONTH', v.sale_date) extrai o número do mês de sale_date e restringe a carga às vendas do período desejado, = 01 para janeiro, = 02 para fevereiro, = 03 para março e > 03 para os meses restantes.

6.3.3. Simulação de mudança de status e processamento do histórico.

Para demonstrar o versionamento SCD Tipo 2, foram aplicadas duas atualizações diretas em Relational.customers.

- **Após a carga de janeiro:** UPDATE Relational.customers SET customer_status = 'Gold' WHERE id_customer BETWEEN 1 AND 5. O atributo customer_status dos clientes de ID 1 a 5 foi promovido para 'Gold';
- **Após a carga de março:** UPDATE Relational.customers SET customer_status = 'Platinum' WHERE id_customer = 3. O atributo customer_status do cliente de ID 3 foi promovido para 'Platinum';
- A cada alteração, o bloco de carga das dimensões (CTE S + CTE UPD + INSERT) foi re-executado. O mecanismo detecta a divergência em customer_status, encerra o registro anterior preenchendo validity_end_date com current_date e insere uma nova linha com validity_start_date igual a current_date e validity_end_date igual a NULL, preservando o histórico completo de versões de cada cliente.

6.3.4. Verificação e auditoria do SCD Tipo 2.

Ao longo do roteiro, consultas de auditoria foram executadas para validar o comportamento do SCD Tipo 2. Os SELECTs cruzam fact_sales com dim_customer, e quando necessário com dim_time, via INNER JOIN pelas respectivas SKs, filtrando c.id_customer BETWEEN 1 AND 5. Os pontos abaixo mostram o que as consultas demonstram.

- Os registros de janeiro em fact_sales apontam para as SKs antigas dos clientes 1 a 5, ou seja, para a versão em que customer_status ainda não era 'Gold'. O vínculo histórico é preservado na fato mesmo após a atualização na dimensão;
- Os registros de fevereiro em diante apontam para as SKs novas, refletindo o customer_status vigente no momento em que cada mês foi carregado;
- Uma consulta adicional com filtro t.time_month = 3 confirma que nenhum dos clientes de ID 1 a 5 realizou compras em março, validando que a fato não gerou registros para esse grupo naquele período.

6.4. Cubo.

Como o pgAdmin 4 não é um servidor OLAP nativo, foi criada a tabela desnormalizada Dimensional.sales_cube por meio do arquivo 4. Cube.sql. Ela une a tabela fato a todas as dimensões em uma única estrutura plana, preparada para consumo direto pelo Power BI.

- O bloco SELECT escolhe apenas os atributos descritivos e métricas relevantes para análise. As SKs (seller_key, customer_key, product_key, time_key), os IDs de origem e os campos de validade (validity_start_date, validity_end_date) são deliberadamente excluídos, são chaves técnicas de controle interno, sem valor analítico. Os atributos selecionados são: customer_name, customer_state, customer_gender e customer_status de dim_customer; product_name de dim_product; time_date, time_day, time_month, time_year e time_quarter de dim_time; seller_name de dim_seller; e as métricas quantity, unit_price, total_price e discount de fact_sales;
- O bloco INTO Dimensional.sales_cube cria fisicamente a tabela de destino no schema Dimensional, consolidando em uma única estrutura todos os dados necessários para as análises no Power BI;
- O bloco FROM parte de Dimensional.fact_sales fv e conecta todas as dimensões por INNER JOIN: dim_customer dc via fv.customer_key = dc.customer_key; dim_product dp via fv.product_key = dp.product_key; dim_time dt via fv.time_key = dt.time_key; e dim_seller dv via fv.seller_key = dv.seller_key. O uso de INNER JOIN em todas as junções garante que apenas combinações com correspondência em todas as dimensões componham o cubo, qualquer registro sem dimensão associada é excluído do resultado.

6.5. KPI (indicador de performance).

Para medir a receita realizada da JJBike em relação às metas mensais, foi criada a tabela Dimensional.kpi pelo arquivo 5. KPI.sql. O script é composto por quatro estruturas principais:

- O bloco SELECT define as três colunas que comporão a tabela: dt.time_month aliado como month, que identifica o mês de referência; SUM(fv.total_price) aliado como total_revenue, que agrega a receita efetivamente realizada somando total_price de todos os registros de fact_sales do respectivo mês; e um bloco CASE dt.time_month que atribui um valor de meta fixo para cada mês, de R\$ 220.000 em janeiro a R\$ 270.000 em dezembro,, aliado como target;
- O operador de conversão ::numeric, posicionado após o END do CASE, faz com que a coluna target seja consolidada com o mesmo tipo numérico de precisão fixa de total_price em fact_sales. Sem essa conversão, o PostgreSQL trataria os valores do CASE como inteiros, causando incompatibilidade de tipo ao calcular percentuais de atingimento no Power BI;
- O bloco INTO Dimensional.kpi cria fisicamente a tabela de destino no schema Dimensional, armazenando a receita realizada e a meta lado a lado para cada mês;
- O bloco FROM parte de Dimensional.fact_sales fv e realiza um INNER JOIN com Dimensional.dim_time dt pela condição fv.time_key = dt.time_key, relacionando cada registro da fato ao seu respectivo mês em dim_time. O GROUP BY dt.time_month consolida todas as vendas de cada mês em uma única linha, e o ORDER BY dt.time_month ASC organiza o resultado em ordem cronológica crescente.

7. Artefatos - Power BI.

As tabelas do banco de dados Dimensional, criadas no pgAdmin 4, foram conectadas ao Power BI.

7.1. Relatórios.

7.1.1. Transformações (T) nas tabelas.

Em fact_sales, foi criada uma coluna chamada discount_percent (com duas casas decimais), com a seguinte fórmula:

- discount_percent = TRUNC(('dimensional fact_sales'[discount] / 'dimensional fact_sales'[total_price]) * 100, 2);

Em dim_customer, foi criada uma coluna chamada full_location, com a seguinte fórmula:

- location_map =
SWITCH(
 'dimensional dim_customer'[customer_state],
 "AC", "Acre",
 "AL", "Alagoas",
 "AM", "Amazonas",
 "AP", "Amapa",
 "BA", "Bahia",
 "CE", "Ceara",
 "DF", "Federal District",
 "ES", "Espirito Santo",
 "GO", "Goias",
 "MA", "Maranhao",
 "MG", "Minas Gerais",
 "MS", "Mato Grosso do Sul",
 "MT", "Mato Grosso",
 "PA", "Para",
 "PB", "Paraiba",
 "PE", "Pernambuco",
 "PI", "Piauhi",
 "PR", "Parana",
 "RJ", "Rio de Janeiro",
 "RN", "Rio Grande do Norte",
 "RO", "Rondonia",
 "RR", "Roraima",
 "RS", "Rio Grande do Sul",
 "SC", "Santa Catarina",
 "SE", "Sergipe",
 "SP", "Sao Paulo",
 "TO", "Tocantins",
 "Unknown"
) & ", Brazil".

7.1.2. Métricas.

Em dim_kpi, foram criadas três métricas para que, no relatório de KPI, o card tenha comportamento dinâmico de acordo com o que estiver selecionado nos botões do button slicer:

- measure_target =
IF(
 ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),


```

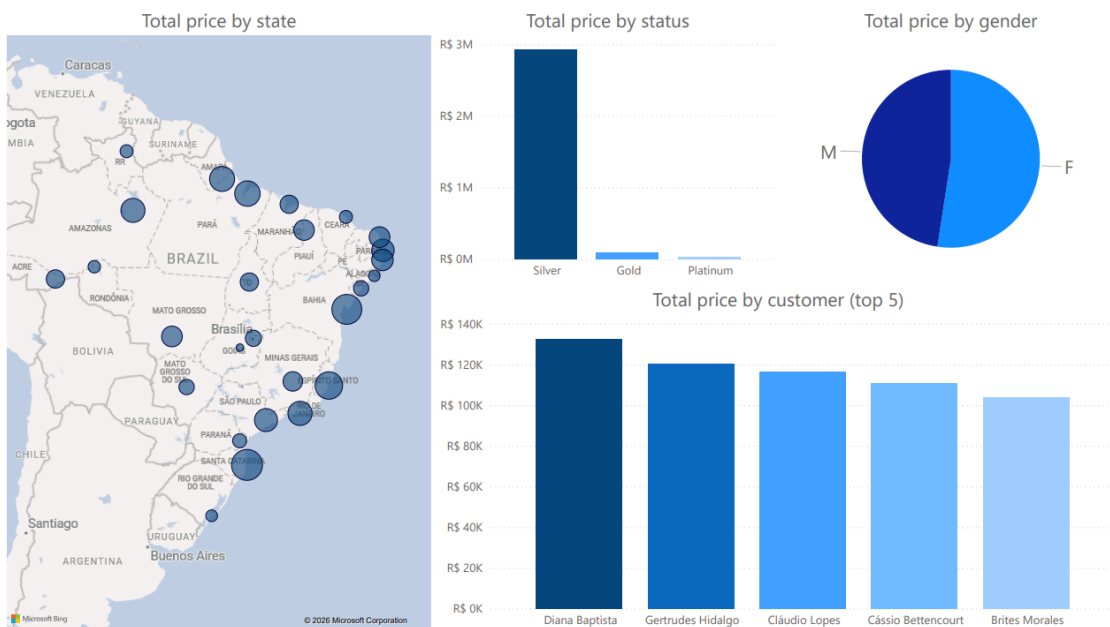
SUM('dimensional kpi'[target]),
0
).
• measure_total_revenue =
IF(
    ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional
dim_time'[time_year]),
SUM('dimensional kpi'[total_revenue]),
0
).

```

7.2.1. Relatórios criados.

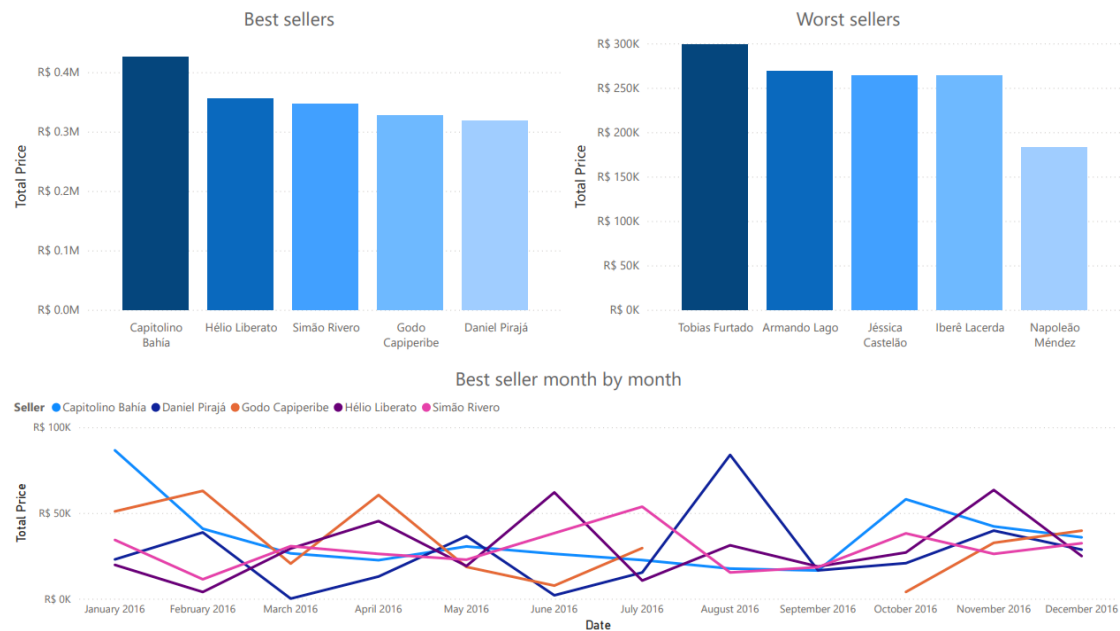
(Clientes)

O relatório de clientes apresenta: somatório de valor total por estado, somatório de valor total por status, somatório de valor total por gênero e somatório de valor total por cliente (top 5).



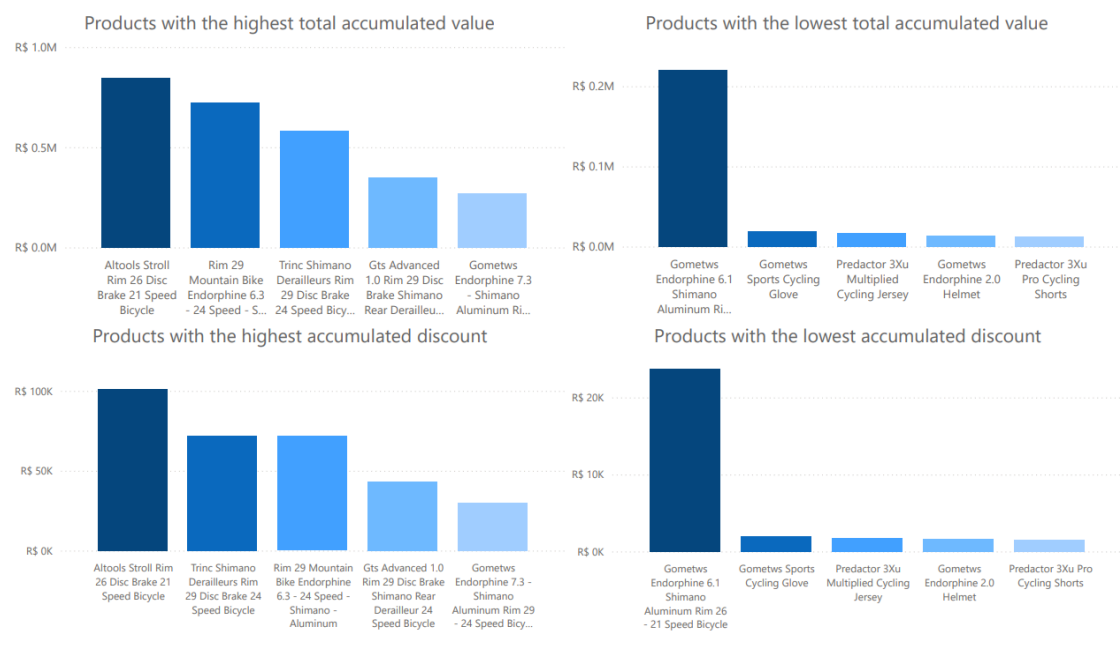
(Vendedores)

O relatório de vendedores apresenta: somatório de valor total por vendedor (top 5 melhores vendedores, top 5 piores vendedores e top 5 melhores vendedores comparados mês a mês).



(Produtos)

O relatório de produtos apresenta: somatório de valor total por produto (top 5 de produtos mais vendidos e top 5 de produtos menos vendidos) e somatório de desconto por produto (top 5 de produtos com mais desconto acumulado e top 5 de produtos com menos desconto acumulado).



7.2. Dashboards.

Foram criados os seguintes dashboards:

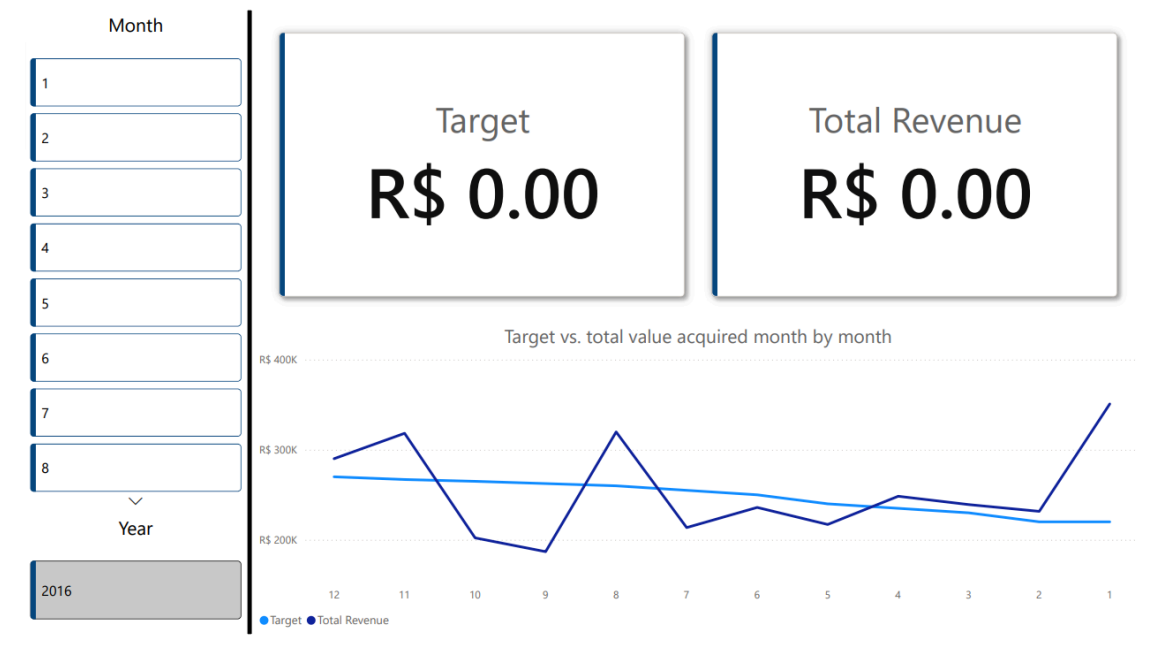
(Vendas)

O relatório de vendas apresenta: somatório de valor total de produtos vendidos (comparados mês a mês), somatório de quantidade de produtos vendidos (comparados mês a mês), somatório de desconto de produtos vendidos (comparados mês a mês) e cards de produtos, vendedores e clientes para filtragem.



(KPI)

O relatório de KPI apresenta: meta, valor total adquirido, somatório de meta e valor total adquirido (comparados mês a mês) e cards de mês e ano para filtragem.



7.3. Cubo.

7.3.1. Hierarquias criadas a partir das dimensões.

Foram criadas as seguintes hierarquias:

- Geographic Hierarchy: customer_state → customer_name (dim_customer);
- Date Hierarchy: time_year → time_quarter → time_month → time_day (dim_time);
- Observações:

- customer_gender e customer_status devem ficar livres no painel lateral como Atributos de Dimensão Soltos, pois inseridos na Geographic Hierarchy implicaria que são parte de uma subdivisão geográfica;
- dim_seller e dim_product possuem apenas um atributo além de suas chaves, portanto não geraram hierarquias e seus atributos tornaram-se Atributos de Dimensão Soltos.

7.3.2. Representação visual do cubo criado.

Customer State	Quantity	Discount	Total Price	
AC		34	R\$ 11,172.65	R\$ 88,878.08
AL		6	R\$ 4,199.33	R\$ 26,817.61
AM		57	R\$ 15,881.10	R\$ 168,571.45
AP		53	R\$ 16,202.60	R\$ 183,945.80
BA		65	R\$ 18,104.59	R\$ 274,690.93
CE		11	R\$ 9,146.31	R\$ 37,562.17
DF		19	R\$ 14,839.02	R\$ 65,272.18
ES		89	R\$ 12,590.55	R\$ 226,925.84
GO		3	R\$ 858.20	R\$ 7,614.38
MA		38	R\$ 5,149.81	R\$ 87,431.06
MG		31	R\$ 18,853.70	R\$ 102,980.97
MS		19	R\$ 11,094.30	R\$ 57,340.96
MT		44	R\$ 11,279.89	R\$ 119,272.51
PA		82	R\$ 24,476.00	R\$ 190,925.91
PB		51	R\$ 8,791.16	R\$ 143,634.26
PE		42	R\$ 19,477.64	R\$ 129,473.93
PI		44	R\$ 20,851.47	R\$ 116,457.99
PR		22	R\$ 5,322.31	R\$ 45,718.33
RJ		57	R\$ 19,318.44	R\$ 169,913.05
RN		45	R\$ 16,227.19	R\$ 118,559.35
RO		13	R\$ 6,639.07	R\$ 33,213.18
RR		28	R\$ 7,962.75	R\$ 37,392.82
RS		12	R\$ 2,768.23	R\$ 28,466.43
SC		91	R\$ 35,968.90	R\$ 296,803.85
SE		15	R\$ 11,054.02	R\$ 56,151.97
SP		42	R\$ 17,043.43	R\$ 152,112.79
TO		47	R\$ 4,163.74	R\$ 88,034.93
Total		1060	R\$ 349,436.40	R\$ 3,054,162.73

Arquivos gerados disponíveis em '2. Artifacts/'.